

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 2**

**COURSE NAME: INTERNET PROGRAMMING**  
**COURSE CODE: CSA4301**

### **COURSE OUTCOME COVERED**

**CO-1:** Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

**Assessment Tool: Data Interpretation (Medium)**

**Weightage: 20%**

**QUESTIONS: Total Marks: 20**

### **1: HTTP Response Analysis**

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

#### **Questions:**

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

### **2: DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
<head>
```

```

<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>

```

### Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

## 3: GET vs POST in Login System

A login system uses two different methods:

| Method | URL Example                 | Data Visibility | Security Level |
|--------|-----------------------------|-----------------|----------------|
| GET    | /login?user=admin&pass=1234 | Visible in URL  | Low            |
| POST   | /login                      | Hidden in body  | Moderate       |

### Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

## 4: HTML Code Inspection

Analyze the following HTML snippet:

```

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>

```

### Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

## 5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

### Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

## Answers:

### 1. HTTP RESPONSE ANALYSIS

#### Given HTTP Response Header

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive 1.

#### Incorrect or Invalid Fields

The main invalid field is:

Content-Length: -150

The Content-Length header specifies the size of the HTTP message body in bytes. It must be a **nonnegative decimal integer**.

A negative value such as -150 is invalid.

The remaining fields are generally valid:

| Header | Analysis |
|--------|----------|
|--------|----------|

|                            |                            |
|----------------------------|----------------------------|
| HTTP/1.1 200 OK            | Valid success              |
| Date                       | Valid HTTP date format     |
| Content-Type:<br>text/html | Valid media type           |
| Content-Length: -150       | Invalid negative value     |
| Connection: keepalive      | Valid connection directive |

## 2. Issue with Content-Length and Its Impact The

header contains:

Content-Length: -150

This is incorrect because the length of an HTTP response body cannot be negative.

### Possible impacts

- The browser may reject the response.
- The HTTP client may treat the response as malformed.
- The server and client may disagree about the size of the message body.
- The connection may be closed unexpectedly.
- The browser may fail to render the page.
- Security problems may occur if different systems interpret the invalid value differently.

The Content-Length value must represent the actual size of the response body.

For example: Content-Length: 150 if

the body contains exactly 150 bytes.

## 3. Does the Status Code Match the Response Correctness?

The status code is:

HTTP/1.1 200 OK

The 200 OK status code indicates that the request was successfully processed and a valid response was returned.

However, the response contains an invalid header:

Content-Length: -150

Therefore, the status code alone does not guarantee that every part of the response is correct.

**Analysis**

200 OK is appropriate only if the server successfully processed the request.

- The invalid Content-Length makes the response malformed.
- Therefore, the status code may indicate successful processing, but the complete HTTP response is not technically valid.

The error is in the response header rather than necessarily in the requested resource.

#### **4. Corrections**

The corrected header should be:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

The exact value of Content-Length must match the actual number of bytes in the response body.

#### **Conclusion**

The primary error is the negative Content-Length value. It must be replaced with a non-negative value representing the actual response body size. Although 200 OK indicates successful processing, the response headers must also be valid for correct HTTP communication.

## **2. DOM STRUCTURE ANALYSIS**

### **Given DOM Structure**

```
<html>
```

```
<head>
```

```
<title>TestPage</title>
```

```
</head>
```

```
<body>
```

```
<div>
```

```
<p>HelloWorld
```

```
</div>
```

```
</body>
```

```
</html>
```

### **1. Missing or Improperly Closed Elements**

•

The following element is not explicitly closed:

```
<p>HelloWorld
```

The correct form is:

```
<p>HelloWorld</p>
```

The `<div>` element is properly closed:

```
</div>
```

The `<html>`, `<head>`, `<title>`, and `<body>` elements are also properly closed.

## 2. How the Browser Interprets the Structure

HTML browsers use error-recovery algorithms to process imperfect HTML.

When the browser encounters:

```
<p>HelloWorld
```

```
</div>
```

it generally automatically closes the `<p>` element when necessary while constructing the DOM tree.

The browser may internally construct a DOM similar to: html

```
├── head
│   └── title
└── body
    ├── div
    │   └── p
    │       └── "HelloWorld"
```

Therefore, the page may still appear to work even though the source HTML is incomplete.

However, relying on automatic browser correction is not recommended.

## 3. Impact on Rendering and Layout

In this simple example, the visual impact may be small because the browser can automatically correct the missing closing tag.

However, invalid HTML may cause:

- Unexpected DOM structure.
- Incorrect CSS application.
- Unexpected margins and spacing.
- Problems with JavaScript DOM manipulation.

- Accessibility issues.

Different behavior in complex nested layouts.

For example, if the paragraph contained additional elements, the browser might automatically close elements in an unexpected order.

This can cause differences between the intended structure and the actual DOM tree created by the browser.

#### **4. Corrected Valid DOM**

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>TestPage</title>
```

```
</head>
```

```
<body>
```

```
  <div>
```

```
    <p>HelloWorld</p>
```

```
  </div>
```

```
</body>
```

```
</html>
```

This structure is clearer and valid because the paragraph is properly closed.

#### **Conclusion**

Although modern browsers can repair many HTML errors automatically, developers should write valid HTML to ensure predictable DOM construction, consistent rendering, accessibility, and maintainability.

### 3. GET VS POST IN LOGIN SYSTEM

#### Given Comparison

| <b>Meth<br/>od</b> | <b>URL Example</b>          | <b>Data<br/>Visibil<br/>ity</b> | <b>Securi<br/>ty<br/>Level</b> |
|--------------------|-----------------------------|---------------------------------|--------------------------------|
| GET                | /login?user=admin&pass=1234 | Visible in URL                  | Low                            |
| POST               | /login                      | Hidden in request body          | Moderate                       |

#### 1. Differences in Data Transmission

##### GET Method

In a GET request, data is appended to the URL:

/login?user=admin&pass=1234 The data is sent as query parameters.

Characteristics:

- Data is visible in the URL.
- Data may be stored in browser history.
- Data may be recorded in server logs.
- Data may be cached.
- Suitable for retrieving non-sensitive information.

##### POST Method

In a POST request, data is placed in the HTTP request body:

POST /login HTTP/1.1

Content-Type: application/x-www-form-urlencoded The credentials are sent in the body rather than in the URL.

Characteristics:

- Data is not displayed in the URL.
- It is more suitable for submitting information.
- It is not automatically secure without HTTPS.

## **2. More Suitable Method for Login**

POST is more suitable for login authentication.

### **Reason**

Login credentials are sensitive information. Using GET exposes them through:

- Browser history.
- URL sharing.
- Server logs.
- Proxy logs.
- Analytics systems.
- Referrer information in some situations.

POST avoids placing credentials directly in the URL.

However, POST alone does **not encrypt data**.

Therefore, secure login should use:

HTTPS + POST

## **3. Security Risks of GET for Authentication**

Using GET for authentication can cause:

### **1. Credential Exposure Example:**

/login?user=admin&pass=1234

Anyone who can see the URL may see the password.

### **2. Browser History Exposure**

The URL may remain in browser history.

### **3. Server Log Exposure**

Web servers may record complete URLs in log files.

### **4. Accidental Sharing**

A user may accidentally copy and share the URL containing credentials.

### **5. Caching**

The URL may be stored in browser or proxy caches.

### **6. Information Leakage**

•

Sensitive credentials may be exposed through monitoring and analytics systems.

#### **4. Best Practices for Secure Login A**

secure login system should:

1. Use POST for credential submission.
2. Use HTTPS/TLS for encryption.
3. Never place passwords in URLs.
4. Hash passwords securely on the server.
5. Use secure session management.
6. Use HttpOnly and Secure cookies.
7. Implement CSRF protection where applicable.
8. Validate input on the server.
9. Apply rate limiting to prevent brute-force attacks.
10. Avoid displaying detailed login error messages that reveal whether a username exists.

#### **Secure Example**

```
<form action="/login" method="POST">
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" required>

  <label for="password">Password:</label>
  <input type="password" id="password" name="password" required>

  <button type="submit">Login</button>
</form>
```

The form should be submitted over HTTPS.

#### **Conclusion**

POST is more appropriate than GET for login because credentials are not included in the URL. However, HTTPS is essential because POST alone does not encrypt the request.



## 4. HTML CODE INSPECTION

### Given HTML Code

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Sample</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Welcome
```

```
  <p>This is a paragraph
```

```
  
```

```
</body>
```

```
</html>
```

### 1. Missing Closing Tags

The following tags are not explicitly closed:

#### Heading

```
<h1>Welcome
```

Correct:

```
<h1>Welcome</h1>
```

#### Paragraph

```
<p>This is a paragraph Correct:
```

```
<p>This is a paragraph</p>
```

The `<img>` element does not require a separate closing tag in HTML5 because it is a **void element**.

## 2. Issues with the <img> Tag

The image element is:

```

```

The main issue is the missing alt attribute.

A better version is:

```

```

### Importance of alt

The alt attribute:

- Helps screen readers describe the image.
- Improves accessibility.
- Displays alternative text if the image cannot load.
- Provides meaningful information about the image.

The src path must also point to a valid image file.

## 3. How Browsers Handle These Errors

Modern browsers use error-recovery mechanisms.

They may automatically infer:

```
<h1>Welcome</h1>
```

```
<p>This is a paragraph</p>
```

Therefore, the page may still render.

However, automatic correction can result in:

- Unexpected DOM structures.
- Unpredictable CSS behavior.
- JavaScript selection problems.
- Accessibility problems.
- Difficulty maintaining the code.

Therefore, developers should explicitly close normal container elements.

## 4. Corrected HTML Code

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph.</p>
  
</body>
</html>
```

## Conclusion

The main errors are the missing closing tags for `<h1>` and `<p>`, along with the missing `alt` attribute in the `<img>` element. The corrected code improves validity, accessibility, and maintainability.

## 5. BROWSER-SERVER COMMUNICATION FLOW

### Given Sequence

1. User enters a URL
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

### 1. Interpretation of Each Step

#### Step 1: User Enters a URL

The user enters a URL such as:

`https://www.example.com`

The browser analyzes:

- Protocol: HTTPS
- Domain name: `www.example.com` • Requested resource: /

#### Step 2: DNS Lookup Occurs

The browser needs the server's IP address.

DNS converts:

www.example.com into

an IP address such as:

192.0.2.10

The browser can then locate the server.

### **Step 3: HTTP Request Is Sent**

After obtaining the IP address, the browser sends a request.

Example:

GET / HTTP/1.1

Host: www.example.com

For HTTPS, a secure TLS connection is established before the HTTP communication.

### **Step 4: Server Processes the Request**

The web server:

1. Receives the request.
2. Identifies the requested resource.
3. Performs application processing if necessary.
4. May access databases or other services.
5. Creates a response.

### **Step 5: Response Is Returned**

The server sends an HTTP response:

HTTP/1.1 200 OK

Content-Type: text/html

The response may contain HTML content.

### **Step 6: Browser Renders the Page**

The browser:

1. Parses the HTML.
2. Creates the DOM.
3. Downloads CSS files.
4. Downloads JavaScript and images.
5. Creates the CSSOM.
6. Combines the DOM and CSSOM.

7. Paints the final page on the screen.

## **2. Locations Where Delays Can Occur**

Delays may occur at several stages:

### **DNS Delay**

Slow DNS resolution can delay the request.

### **Network Delay**

The request may take time to travel between the client and server.

### **TLS Delay**

HTTPS requires secure connection establishment.

### **Server Processing Delay**

The server may take time to:

- Execute application code.
- Query a database.
- Process an API request.

### **Response Transfer Delay**

Large files take longer to download.

### **Browser Rendering Delay**

Complex HTML, CSS, JavaScript, and images can slow rendering.

## **3. Role of DNS**

DNS means **Domain Name System**.

Its main role is to translate human-readable domain names into IP addresses.

Example: www.example.com

↓

IP Address

Without DNS, users would need to remember numerical IP addresses.

DNS may also help with:

- Load balancing.
- Server distribution.
- Redundancy.
- Finding mail servers through DNS records.

## Communication Example

User



www.example.com



DNS Resolver



IP Address



Web Server

## 4. Optimizations to Improve Performance

### 1. Use DNS Caching

Caching reduces repeated DNS lookup time.

### 2. Use a Content Delivery Network (CDN)

A CDN serves static content from servers closer to users.

### 3. Enable HTTP/2 or HTTP/3

Modern HTTP protocols improve connection efficiency and resource loading.

### 4. Compress Responses

Use compression techniques such as gzip or Brotli to reduce file size.

### 5. Optimize Images Use:

- Correct image dimensions.
- Modern formats.
- Image compression.

### 6. Use Browser Caching

Frequently used resources can be stored locally.

### 7. Reduce Server Processing Time Optimize:

- Database queries.
- Backend code.
- API response time.

### 8. Minify CSS and JavaScript

Removing unnecessary spaces and characters reduces file size.

### 9. Reduce Unnecessary Requests

Combining or optimizing resources can reduce network overhead.

## **10. Use Persistent Connections**

Persistent connections reduce the need to establish a new connection for every resource.

### **Overall Flow**

User



Enter URL



DNS Lookup



IP Address Found



HTTP/HTTPS Request



Server Processing



HTTP Response



HTML/CSS/JS/Image Downloads



Browser Rendering



Web Page Displayed

### **Conclusion**

Browser-server communication involves multiple stages, beginning with URL entry and DNS resolution and ending with browser rendering. Performance can be improved by reducing DNS, network, server-processing, file-transfer, and rendering delays.

The major conclusions are:

- Content-Length must always be a valid non-negative value.
- HTML elements should be properly nested and closed.
- POST is more suitable than GET for login credentials.
- Images should contain meaningful alt attributes.

- DNS translates domain names into IP addresses.
- Performance depends on optimizing every stage of the browser-server communication cycle.

Correct application of these concepts helps developers build reliable, accessible, secure, and efficient web applications.

***Code of Conduct:***

*I T Ganeswari(192372183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student: T Ganeswari***

**RUBRICS FOR CALCULATION:**

| Criteria                | Weightage | Level 4 (Exemplary)                                                       | Level 3 (Proficient)                                 | Level 2 (Developing)                     | Level 1 (Beginning)                       |
|-------------------------|-----------|---------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------|-------------------------------------------|
| Understanding of Data   | 25%       | Accurately interprets all data patterns, trends, and relationships        | Interprets data correctly with minor gaps            | Partial understanding; misses key trends | Misinterprets or fails to understand data |
| Analysis                | 25%       | Provides deep analysis with strong logical reasoning and justification    | Provides reasonable analysis with some justification | Basic analysis with weak reasoning       | No meaningful analysis provided           |
| Application of Concepts | 20%       | Effectively applies relevant concepts/tools to interpret data accurately  | Applies concepts with minor errors                   | Limited application; requires guidance   | Unable to apply concepts to data          |
| Conclusion              | 20%       | Draws clear, meaningful conclusions with strong insights and implications | Provides valid conclusions with moderate insights    | Conclusions are vague or weak            | No clear or valid conclusions             |

|         |     |                                                                                  |                                   |                                   |                                               |
|---------|-----|----------------------------------------------------------------------------------|-----------------------------------|-----------------------------------|-----------------------------------------------|
| Clarity | 10% | Presents interpretation clearly using proper structure, visuals, and terminology | Generally clear with minor issues | Lacks clarity or proper structure | Poor presentation and difficult to understand |
|---------|-----|----------------------------------------------------------------------------------|-----------------------------------|-----------------------------------|-----------------------------------------------|